

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

Department of Information Technology

Fall 2026-27

Operating Systems Lab BITE303P

PROJECT PROPOSAL

KernelLeakGuard: Hybrid Static Analysis and Runtime Verification of Memory Leaks in Linux Kernel Early-Exit Paths

Submitted by

Om Gunjan Gupta 24BIT0316

Abhishek Kumar Singh 24BIT0309

Aarav Jain 24BIT0023

Under the Guidance of

Dr. ARIVUSELVAN K

Associate Professor, SCORE

VIT, Vellore.

Fall 2026-27

PROJECT PROPOSAL

1. Problem Statement

Linux kernel routines exit early when an allocation or a device operation fails. Each early exit must release everything acquired so far; a forgotten kfree() leaks kernel memory permanently. Because such paths run only when something fails, testing rarely executes them, and static analysers report many warnings that are not real leaks. The problem addressed is: find memory leaks on kernel early-exit paths and automatically verify each finding by reproducing the failure in an isolated virtual machine, so that only evidence-backed leaks reach the developer.

2. Objectives

1. Implement an LLVM-based static analyser (C++) that detects allocations neither freed nor handed over on paths to a function exit, inspired by MLEE (USENIX ATC 2021).
2. Identify, for each warning, the failing call that leads onto the leaking path.
3. Build an isolated QEMU/KVM Linux test environment with kmemleak and fault injection.
4. Inject the predicted failure automatically and classify each warning as confirmed, not reproduced or inconclusive using kmemleak evidence.
5. Evaluate on a benchmark of kernel modules with ground truth (static vs hybrid vs dynamic-only) and provide a dashboard.

3. Scope

In scope: loadable kernel modules compiled with clang for Linux 6.18 LTS; kmalloc, vmalloc, kvmalloc and slab-cache allocations and their release functions; failures injectable with failslab, fail_function and fail_usercopy; a benchmark corpus written by the team. Out of scope: analysis of the complete kernel, non-memory resources, hardware-dependent drivers, and recreating MLEE's full research system.

4. Literature / Background

MLEE (Wang, 2021) detects missing releases on early-exit paths statically and found 120 new Linux leaks but leaves confirmation to developers. Hector (Saha et al., DSN 2013), K-MELD (Emamdoost et al., NDSS 2021) and HERO (Wu et al., USENIX Security 2021) are further static approaches. FIFUZZ (Jiang et al., USENIX Security 2020) and syzkaller with kmemleak

execute error paths dynamically but only find leaks on paths they reach and give no static root cause. Linux provides kmemleak (a tracing mark-and-sweep leak detector) and a fault-injection framework (failslab, fail_function, fail_usercopy, per-task fail-nth). The proposed work combines static prediction with targeted fault injection and kmemleak.

5. Methodology

1. **Static analysis:** compile modules to LLVM IR with the exact kbuild flags; track allocations through aliases and stack objects; classify releases and ownership transfers; compute function summaries; search paths from each allocation to every exit, pruning infeasible paths; report exit, error code and trigger calls.
2. **Test environment:** custom Linux 6.18.55 kernel with kmemleak and fault injection, booted in QEMU/KVM from a minimal initramfs with a shared folder.
3. **Fault-point mapping:** run each routine with fail-nth = 1, 2, 3 ... and map each injected fault to a source line from its stack trace.
4. **Targeted verification:** inject the fault that fails the predicted trigger call, scan kmemleak, attribute leaked objects to allocation sites, decide the verdict.
5. **Evaluation:** precision, recall and cost against ground truth; ablation of analysis features; comparison with dynamic-only baselines.

6. Tools / Software

Table 6.1: Tools and software

Purpose	Tool
Static analysis	C++17, LLVM/Clang 21
Kernel and modules	Linux 6.18.55 LTS, kbuild, kmemleak, fault injection
Isolation	QEMU with KVM, WSL2 on Windows 11, busybox initramfs
Runtime agent	C (statically linked)
Automation, evaluation, dashboard	Python 3, Streamlit, matplotlib
Version control	Git

7. Work Plan

Table 7.1: Work plan

Phase	Activities	Deliverable
1	Literature study (MLEE, kmemleak, fault injection); environment setup; kernel build	Proposal; bootable test VM

2	Benchmark corpus with ground truth; IR extraction; analyser core (aliases, path search)	First static findings
3	Ownership transfer, summaries, branch correlation; in-guest agent; fault-point sweep	Review-I: working prototype (60-70%)
4	Targeted verification, kmemleak attribution, verdicts; baselines; dashboard	End-to-end pipeline
5	Evaluation, ablation, repeated runs; report, user manual, presentation	Review-II, final report and PPT

8. Team Member Responsibilities

Table 8.1: Responsibilities

Member	Register No.	Responsibilities
Om Gunjan Gupta	24BIT0316	Static analyser (C++/LLVM): IR preparation, alias and ownership analysis, function summaries, path search, trigger identification
Abhishek Kumar Singh	24BIT0309	Kernel configuration and build, QEMU/KVM environment, klg-agent fault-injection driver, kmemleak protocol
Aarav Jain	24BIT0023	Benchmark corpus and ground truth, Python orchestrator and verifier, evaluation, dashboard